

Instrukcja obsługi robota rysującego YuMi



Spis treści

1	Instrukcja krok po kroku	4
1.1	Przygotowanie do kalibracji	4
1.1.1	Ustawienie robota	4
1.1.2	Obsługa błędów	4
1.1.3	Przejsie w tryb ręczny	5
1.2	Kalibracja ramion	6
1.2.1	Kalibracja pierwszego ramienia	6
1.2.2	Kalibracja drugiego ramienia	7
1.2.3	Pointer to Main	7
1.3	Połączenie z komputerem i tryb AUTO	7
1.3.1	Połączenie	7
1.3.2	Tryb automatyczny	8
1.4	Uruchamianie aplikacji	8
1.4.1	Na robocie	8
1.4.2	Na komputerze	8
1.5	Obsługa aplikacji graficznej	8
1.5.1	Ustawienia połączenia	8
1.5.2	Zdjęcie	9
1.5.3	Wybór trybu	10
1.5.4	Rozpoczęcie rysowania	10
1.6	Komendy głosowe	11

2	Najczęstsze błędy i ich rozwiązania	11
2.1	Problemy z połączeniem robota	11
2.2	Błędy podczas kalibracji	12
2.3	Problemy z kamerą	12
2.4	Błędy generowania obrazu przez OpenAI	12
2.5	Problemy z przetwarzaniem obrazu	13
2.6	Błędy podczas rysowania	13
2.7	Problemy z rozpoznawaniem głosu	13
2.8	Inne problemy	14
3	Alarmy do zaakceptowania	14
4	FAQ	15
4.1	Jak działa cały proces tworzenia rysunku przez robota?	15
4.2	Jakie są metody przetwarzania obrazu?	15
4.3	Po co przetwarzamy zdjęcia w chmurze z AI?	15
4.4	Jak dokładnie przechodzimy z binarnego obrazu na kontury?	15
4.5	Jak działa tworzenie trasy rysowania wzdłuż konturu?	16
4.6	Jak system inteligentnie dzieli pracę między dwa ramiona robota?	16
4.7	Jak działa komunikacja między komputerem a robotem?	16
5	Szczegółowa struktura kodu	17
5.1	Pliki główne aplikacji	17
5.1.1	simple_gui.py - Interfejs użytkownika	17
5.1.2	robot_drawer.py - Główny koordynator	17
5.1.3	robot_communication.py - Komunikacja TCP/IP	18
5.1.4	image_processor.py - Przetwarzanie obrazu	18
5.1.5	coordinate_transformer.py - Transformacja współrzędnych	18
5.1.6	voice_commands.py - Rozpoznawanie głosu	19
5.1.7	convert_to_lineart.py - Integracja z OpenAI	19
5.2	Pliki konfiguracyjne i dane	19
5.2.1	robot_gui_config.json - Konfiguracja aplikacji	19
5.2.2	line_art_catalog/ - Archiwum wygenerowanych obrazów	20
5.3	Pliki RAPID dla robota	20
5.3.1	Right_arm.mod - Program prawego ramienia	20
5.3.2	Left_arm.mod - Program lewego ramienia	20
6	Dostrajanie algorytmów przetwarzania obrazu	21
6.1	Parametry detekcji krawędzi	21
6.1.1	Threshold - parametry binaryzacji	21
6.1.2	Canny - parametry detekcji	21

6.1.3	Adaptive - parametry adaptacyjne	22
6.2	Parametry upraszczania konturów	22
6.2.1	Parametr epsilon approxPolyDP	22
6.2.2	Minimalna długość konturu	23
6.3	Parametry wygładzania ścieżek	23
6.3.1	Parametry krzywej Béziera	23
7	Modyfikacja współrzędnych i bezpieczeństwa robota	24
7.1	Parametry bezpieczeństwa dual-arm	24
7.1.1	Bufor kolizyjny	24
7.1.2	Zakazana strefa lewego ramienia	24
7.2	Parametry ruchu robota	25
7.2.1	Prędkości ruchu	25
7.2.2	Precyzja pozycjonowania	25
7.3	Parametry systemu współrzędnych	25
7.3.1	Początek układu współrzędnych	25
7.3.2	Skalowanie współrzędnych	26
8	Debugowanie i rozwiązywanie problemów	26
8.1	Debugowanie połączenia z robotem	26
8.1.1	Sprawdzanie dostępności robota	26
8.1.2	Testowanie portów komunikacyjnych	27
8.1.3	Debugowanie timeout'ów	27
8.2	Debugowanie przetwarzania obrazu	27
8.2.1	Wizualizacja kroków przetwarzania	27
8.2.2	Zapisywanie obrazów pośrednich	27
8.3	Debugowanie ruchu robota	28
8.3.1	Rejestrowanie komend wysyłanych do robota	28
8.3.2	Testowanie pojedynczych ruchów	28
8.4	Debugowanie trybu dual-arm	28
8.4.1	Wizualizacja przydziału konturów	28
8.4.2	Testowanie synchronizacji ramion	28
9	Rozszerzanie funkcjonalności systemu	29
9.1	Dodawanie nowych metod detekcji krawędzi	29
9.1.1	Implementacja nowej metody	29
9.1.2	Integracja z interfejsem	29
9.2	Dodawanie nowych komend głosowych	29
9.2.1	Rozszerzenie słownika komend	29
9.2.2	Implementacja akcji komendy	30
9.3	Dodawanie nowych stylów rysowania	30

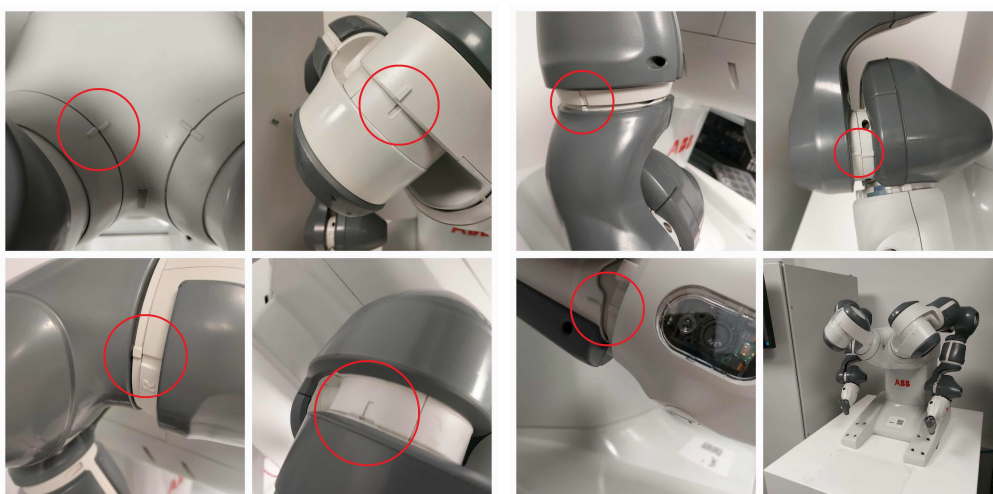
9.3.1	Implementacja stylu w OpenAI	30
9.3.2	Integracja z interfejsem	30

1 Instrukcja krok po kroku

1.1 Przygotowanie do kalibracji

1.1.1 Ustawienie robota

Ustaw wyłączonego robota w pozycji do kalibracji tak, jak na zdjęciach (widoczne ząbki i wskaźniki).



1.1.2 Obsługa błędów

Potwierdź wszystkie błędy (*Acknowledge errors*).

Error Not Acknowledged

38200 Battery backup lost

Event Log - Event Message

Event Message 38200

2025-09-12 13:01:11

Battery backup lost

Description

The battery backup to serial measurement board (SMB) 1 in the robot connected to drive module 1 on measurement link 1 has been lost.

Consequences

When the SMB battery power supply is interrupted, the robot will lose the revolution counter data. This warning will also repeatedly be logged.

Probable causes

This may be due to an SMB battery that is discharged or not connected. For some robot models, the SMB battery power is supplied through a jumper in the robot.

Show Log

Acknowledge

1.1.3 Przejście w tryb ręczny

Uruchom silniki i przejdź w tryb ręczny. Jeżeli wyskakują błędy, zaakceptuj je i ponów próbę.

Auto

Waiting for motors on after e-stop

14000-500767 (192.168.1.1)

Stopped (3 of 3) (Speed 100%)

RobotWare

Operator Panel

Motor On / Off

Off

Note: The Three-position enabling device is inactive

Operator Mode

Auto

Manual

1

2

3

4

5

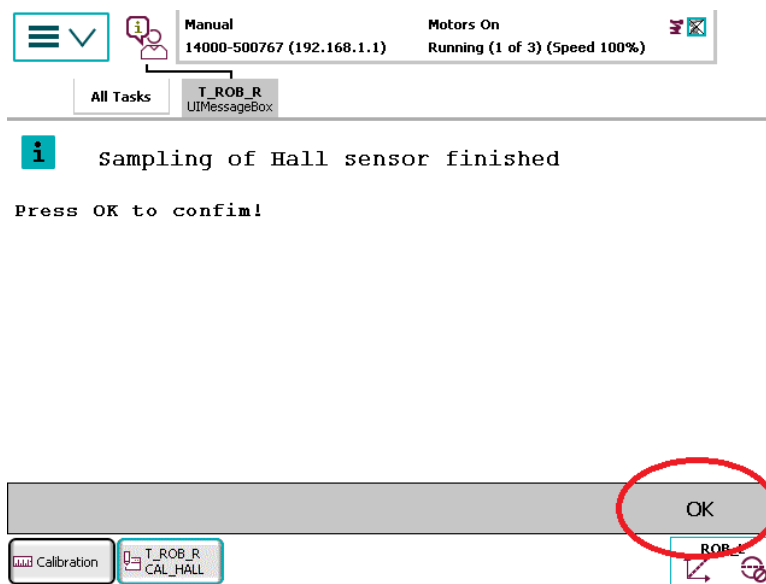
1.2 Kalibracja ramion

1.2.1 Kalibracja pierwszego ramienia

Włącz kalibrację pierwszego ramienia. Wszystkie wyskakujące błędy należy akceptować. Następnie naciśnij przycisk oznaczony trójkątem na flexpendancie, aby rozpocząć aplikację.

The screenshots illustrate the following steps:

- Menu Navigation:** The user navigates through the main menu to the 'Calibration' option, which is highlighted with a red circle and the number 2.
- Unit Selection:** The user selects the mechanical unit 'ROB_R' for calibration. The status 'Revolution counters not updated' is shown for both 'ROB_R' and 'ROB_L', with 'ROB_R' highlighted by a red circle and the number 3.
- Calibration Method Selection:** The user selects 'Call Calibration Method' from the 'Manual Method (Advanced)' section, highlighted with a red circle and the number 4.
- Calibration Process:** The process continues through several prompts:
 - Calibration with hall sensors:** The user selects 'Update of revolution counters'.
 - Selection of joint(s) to update:** The user selects '1. [X]' for the first joint.
 - Sampling of Hall sensor starting:** The user presses 'Next' to start the sampling process.
 - Completion:** The user presses 'OK' to complete the calibration.

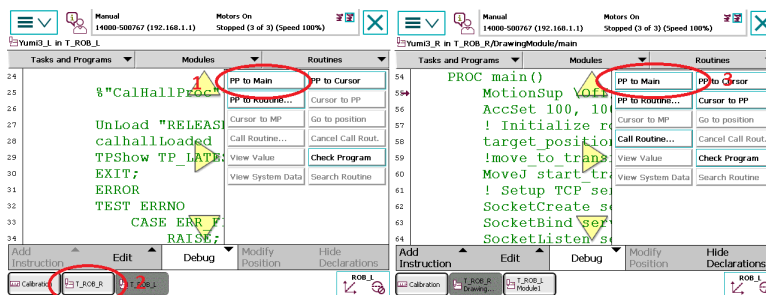


1.2.2 Kalibracja drugiego ramienia

Włącz kalibrację drugiego ramienia analogicznie do pierwszego.

1.2.3 Pointer to Main

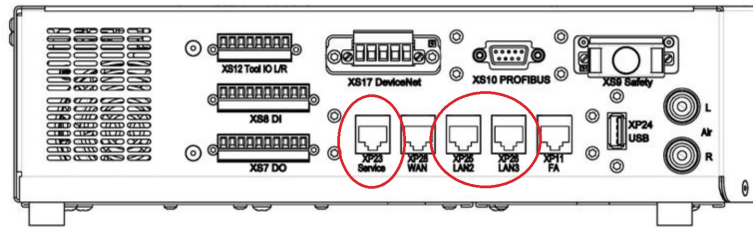
Ustaw *pointer to main* na obu ramionach.



1.3 Połączenie z komputerem i tryb AUTO

1.3.1 Połączenie

Podłącz port LAN robota do komputera kablem ethernetowym.



1.3.2 Tryb automatyczny

Przejdź w tryb automatyczny (tak samo jak wcześniej w manualny, ale bez uruchamiania silników).

1.4 Uruchamianie aplikacji

1.4.1 Na robocie

Uruchom aplikację na robocie. Naciśnij przycisk oznaczony trójkątem na flexpendancie, aby rozpocząć pracę aplikacji.

Uwaga: robot zacznie się ruszać!

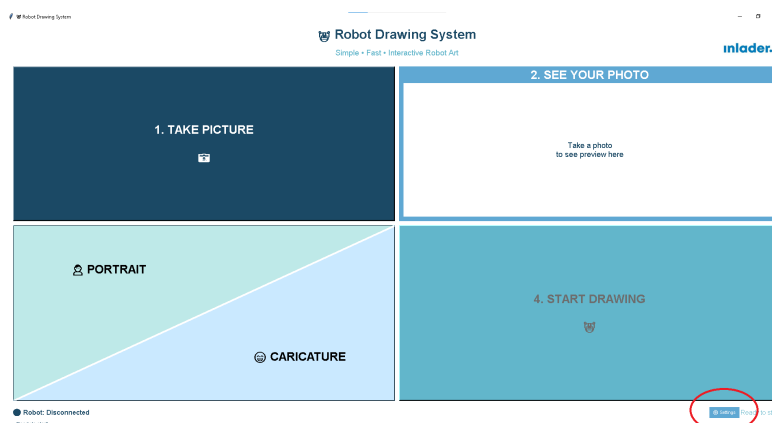
1.4.2 Na komputerze

Uruchom aplikację na komputerze, otwierając plik `simple_gui.py` w Pythonie.

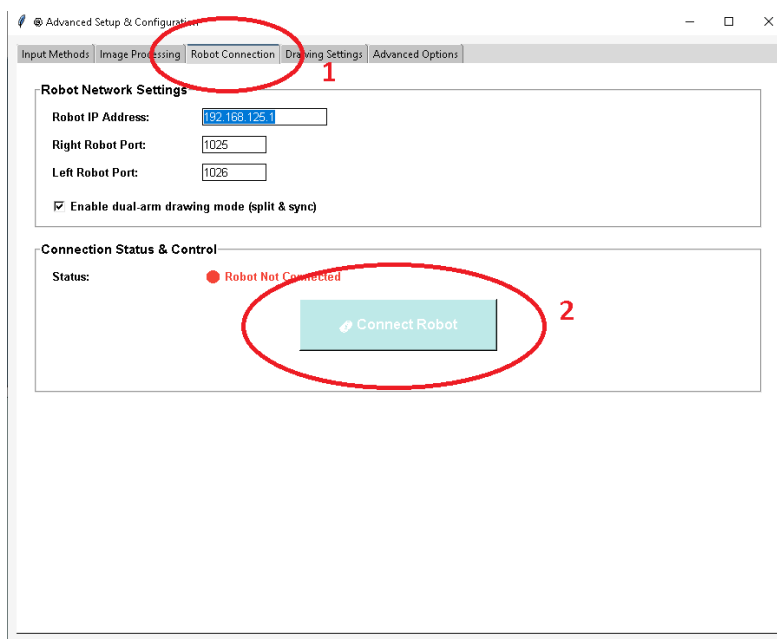
1.5 Obsługa aplikacji graficznej

1.5.1 Ustawienia połączenia

Wejdź w **Settings**.



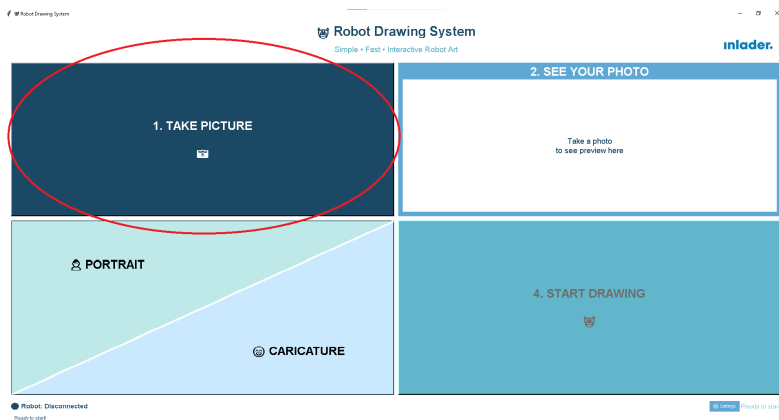
W sekcji **Robot Connection** kliknij *Connect*.



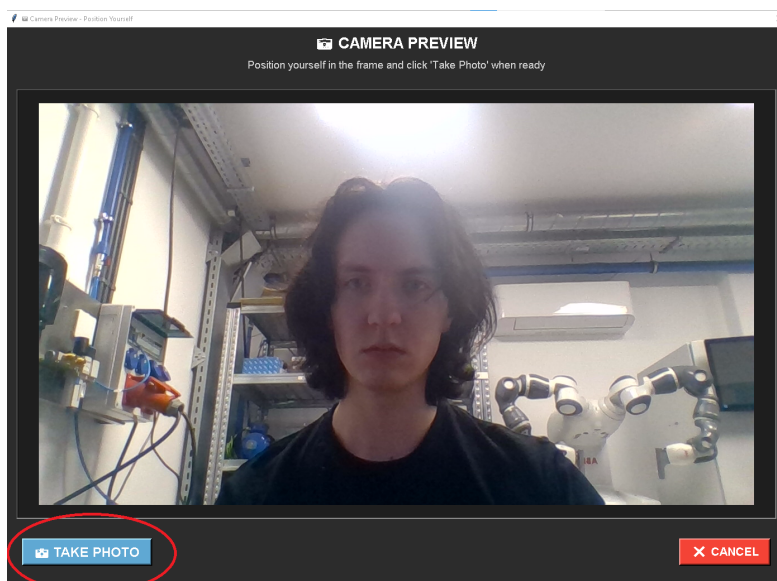
Zamknij ustawienia.

1.5.2 Zdjęcie

Kliknij **Take Picture**.



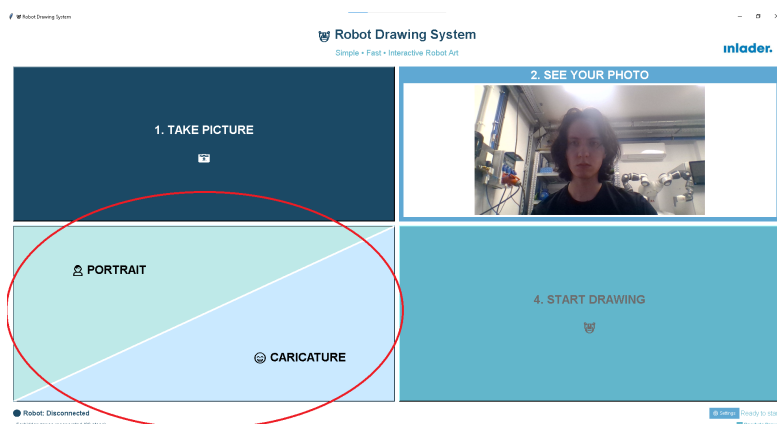
Ustaw się i naciśnij **Take Photo**.



Oceń zdjęcie. Jeśli nie spełnia oczekiwań, wykonaj nowe.

1.5.3 Wybór trybu

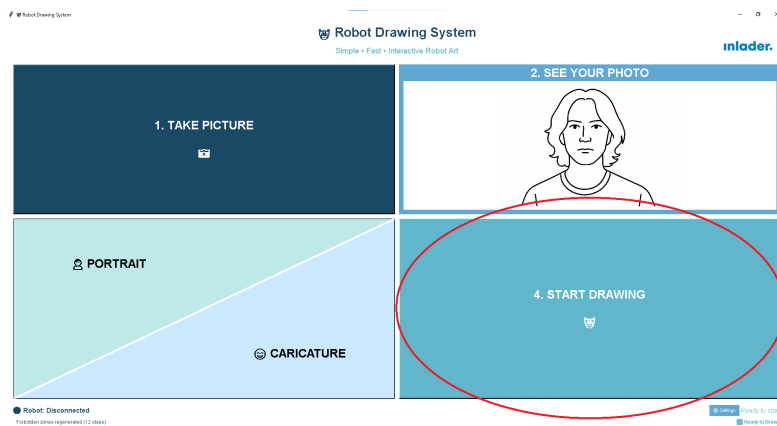
Wybierz tryb: **Portret** lub **Karykatura**.



Obraz zostanie wygenerowany (wymagane stabilne połączenie z Internetem).

1.5.4 Rozpoczęcie rysowania

Naciśnij **Start Drawing**, aby rozpocząć rysowanie.



1.6 Komendy głosowe

- **Kamera** - włącz kamerę
- **Akcja** - zrób zdjęcie
- **Portret** - stwórz portret
- **Karykatura** - stwórz karykaturę
- **Połącz** - połącz z robotem
- **Start** -start
- **Stop** - stop

2 Najczęstsze błędy i ich rozwiązania

2.1 Problemy z połączeniem robota

Błąd: Nie można połączyć się z robotem. Aplikacja wyświetla komunikat o błędzie połączenia.

Rozwiązanie:

- Sprawdź, czy robot jest włączony i w trybie automatycznym.
- Upewnij się, że kabel Ethernet jest prawidłowo podłączony między komputerem a robotem.
- Zweryfikuj adres IP robota (domyślnie 192.168.125.1) i porty (1025 dla prawego ramienia, 1026 dla lewego).

- Sprawdź ustawienia firewall na komputerze – upewnij się, że TCP połączenia na portach 1025 i 1026 są dozwolone.
- Uruchom ponownie aplikację na robocie i komputerze.

2.2 Błędy podczas kalibracji

Błąd: Robot nie kalibruje się lub wyświetla błędy podczas uruchamiania aplikacji.

Rozwiązanie:

- Upewnij się, że robot jest w pozycji kalibracyjnej (widoczne ząbki i wskaźniki na zdjęciach).
- Potwierdź wszystkie błędy na panelu robota przyciskiem "Acknowledge errors".
- Przejdź w tryb manualny, a następnie wróć do trybu automatycznego.
- Ustaw "pointer to main" na obu ramionach.

2.3 Problemy z kamerą

Błąd: Nie można zrobić zdjęcia lub kamera nie działa.

Rozwiązanie:

- Sprawdź, czy kamera jest podłączona do komputera i rozpoznawana przez system (np. w Menedżerze urządzeń w Windows).
- Upewnij się, że inne aplikacje nie używają kamery.
- Spróbuj zmienić indeks kamery w ustawieniach aplikacji (domyślnie 0).
- Jeśli kamera zewnętrzna, sprawdź połączenie USB i zasilanie.
- Zamknij i uruchom ponownie aplikację.

2.4 Błędy generowania obrazu przez OpenAI

Błąd: OpenAI nie generuje obrazu lub zwraca błąd API.

Rozwiązanie:

- Sprawdź połączenie internetowe – wymagane stabilne łącze.
- Upewnij się, że klucz API OpenAI jest prawidłowo skonfigurowany w aplikacji.

- Sprawdź limit wykorzystania API – jeśli przekroczony, poczekaj lub wpłać pieniądze.
- Spróbuj ponownie z tym samym zdjęciem lub wykonaj nowe zdjęcie.
- Jeśli błąd trwa, sprawdź status usług OpenAI.

2.5 Problemy z przetwarzaniem obrazu

Błąd: Obraz po przetworzeniu ma złe kontury lub jest nieczytelny.

Rozwiązanie:

- Wypróbuj inną metodę detekcji krawędzi (threshold, canny lub adaptive) w ustawieniach.
- Dostroj parametry jakości (wyższa jakość dla lepszych wyników, ale dłuższy czas przetwarzania).
- Wykonaj lepsze zdjęcie – upewnij się o dobrym oświetleniu i ostrości.
- Sprawdź, czy obraz nie jest zbyt złożony; uprość tło lub pozycję.

2.6 Błędy podczas rysowania

Błąd: Robot zatrzymuje się podczas rysowania lub występuje kolizja.

Rozwiązanie:

- Włącz tryb manualny i odjedź ramionami w bezpieczne miejsce
- Zwiększ bufor kolizyjny w ustawieniach (domyślnie 40 mm).
- Sprawdź marginesy bezpieczeństwa (domyślnie 10 mm od krawędzi).
- Jeśli timeout, zwiększ timeout w ustawieniach lub sprawdź połączenie.
- W przypadku zatrzymania, użyj przycisku awaryjnego stop na robocie.

2.7 Problemy z rozpoznawaniem głosu

Błąd: VOSK nie rozpoznaje poleceń głosowych.

Rozwiązanie:

- Upewnij się, że model VOSK (vosk-model-small-pl-0.22) jest pobrany i w odpowiednim folderze.

- Mów wyraźnie po polsku, w cichym środowisku.
- Dostroj czułość mikrofonu w ustawieniach systemu.
- Jeśli nie działa, użyj komend tekstowych zamiast głosowych.
-

2.8 Inne problemy

Błąd: Aplikacja się zawiesza lub wyświetla błędy Python.

Rozwiązanie:

- Sprawdź w terminalu jakie błędy wyskakują.
- Sprawdź, czy wszystkie wymagane biblioteki są zainstalowane (np. OpenCV, Matplotlib, Shapely).
- Uruchom aplikację z wiersza poleceń, aby zobaczyć szczegóły błędów.
- Sprawdź dostępność pamięci RAM – przetwarzanie dużych obrazów wymaga zasobów.
- Zaktualizuj Python do wersji 3.8+.
- Jeśli błąd trwa, zrestartuj komputer i spróbuj ponownie.

3 Alarmy do zaakceptowania

Naciśnij "Acknowledge" zawsze jak się pojawiają.

- **20032** Rev counter not updated
- **38200** Battery backup lost
- **50091** Restart not possible
- **50441** Low voltage on battery inputs
- **50490** Measurement error detected
- **71367** No communication with I/O device

4 FAQ

4.1 Jak działa cały proces tworzenia rysunku przez robota?

Cały proces rozpoczyna się od wykonania zdjęcia za pomocą kamery podłączonej do komputera. Następnie zdjęcie jest wysyłane poprzez API OpenAI z wybranym promptem (portret lub karykatura), aby wygenerować nowy obraz w wybranym stylu. Zwrocony obraz jest przetwarzany przez jedną z trzech metod detekcji krawędzi (threshold, canny lub adaptive), upraszczany do konturów, obracany i optymalizowany pod kątem ścieżki rysowania. Na koniec robot ABB YuMi rysuje obraz na podstawie wygenerowanych współrzędnych.

4.2 Jakie są metody przetwarzania obrazu?

System oferuje trzy metody detekcji krawędzi:

- - **Threshold:** Prosta binaryzacja obrazu ze sztywno ustaloną jasnością odcięcia (dobra dla dobrego światła) 1 gdzie jest ciemniej, 0 gdzie jaśniej.
- **Canny:** Znajdywanie zmian jasności w miejscu zmian dajemy 1, gdzie nie ma zmian dajemy 0.
- **Adaptive:** To samo co Threshold, ale sam dobiera parametry (bardziej uniwersalny).

4.3 Po co przetwarzamy zdjęcia w chmurze z AI?

Możemy uzyskać zdecydowanie ładniejsze proste linie żeby stworzyć najlepszy obrazek do rysowania inaczej robot by pracował ekstremalnie długo żeby narysować każdy cień we włosach itp.

4.4 Jak dokładnie przechodzimy z binarnego obrazu na kontury?

Po zastosowaniu jednej z trzech metod detekcji krawędzi (threshold, canny lub adaptive), obraz staje się binarny – czarno-biały, gdzie białe piksele reprezentują krawędzie, a czarne tło. Następnie używamy algorytmu który skanuje obraz i identyfikuje połączone grupy białych pikseli jako kontury. Każdy kontur to lista punktów współrzędnych (x,y), które opisują kształt linii. Kontury są upraszczane za pomocą approxPolyDP() z parametrem epsilon, aby zredukować liczbę punktów bez utraty kształtu – to kluczowe dla wydajności, bo robot nie musi rysować każdego piksela. Na koniec kontury są filtrowane: małe artefakty są usuwane, a duże kontury dzielone na segmenty dla lepszej kontroli.

4.5 Jak działa tworzenie trasy rysowania wzdłuż konturu?

Gdy mamy kontury jako listy punktów, system tworzy trasę rysowania, traktując każdy kontur jak ścieżkę. Punkty są łączone w sekwencje, tworząc linię ciągłą. Dla płynności stosujemy wygładzanie – algorytm Bézier interpoluje punkty, eliminując ostre kąty. Opcjonalnie włączamy TSP (Traveling Salesman Problem), który optymalizuje kolejność rysowania konturów, minimalizując czas przejścia między nimi (np. rysujemy najbliższy kontur następny). Ścieżka jest skalowana do współrzędnych robota (np. 290x210 mm) z marginesami bezpieczeństwa. W trybie dual-arm ścieżki są dzielone między ramiona, a każdy punkt ma przypisany czas ruchu dla synchronizacji.

4.6 Jak system inteligentnie dzieli pracę między dwa ramiona robota?

W trybie dual-arm system używa zaawansowanego algorytmu master-slave assignment, ale uwaga – role master i slave nie są stałe! System dynamicznie decyduje, które ramię zostaje "mózgiem operacji" (masterem), a które "wsparciem" (slave), w zależności od położenia konturów, ale zazwyczaj zamieniają się co kontur. Kluczowe są specjalne strefy: lewe ramię ma zakazaną strefę (0,0 do 130,40 mm), więc zawsze unika tego obszaru, by nie kolidować z podstawą robota. System stosuje bufor kolizyjny (domyślnie 40 mm) – kołowy "bąbel bezpieczeństwa" wokół mastera oraz "ogon" od tego "bąbla" do krawędzi kartki w stronę mastera, do którego slave nie może wejść. Algorytm sprawdza nakładanie się wielokątów konturów z tymi buforami, inteligentnie przypisując zadania, aby uniknąć kolizji. Dodatkowo, kontury są sortowane wg pozycji: bliższe ramię dostaje priorytet. Ramiona między konturami podróżują w kształcie kanciastego U żeby uniknąć kolizji podczas transzycji, także pozwala to ograniczyć czas robotów spędzony bardzo blisko siebie. To wszystko razem pracując pozwala na rysowanie używając dwóch ramion.

4.7 Jak działa komunikacja między komputerem a robotem?

Komunikacja między komputerem a robotem ABB YuMi odbywa się poprzez protokół TCP/IP. Komputer łączy się z robotem na dwóch portach: 1025 dla prawego ramienia i 1026 dla lewego ramienia. Wysyłane są komendy tekstowe, takie jak "MOVE,x,y" do przesunięcia ramienia, "BATCH, $x_1, y_1, x_2, y_2, \dots$ " do wysłania grupy punktów, "PEN UP" i "PEN DOWN" do kontroli narzędzia. W trybie dual-arm synchronizacja ramion jest realizowana za pomocą barier wątków i komend "WAIT". Tryb batch grupuje punkty w paczki (maksymalnie 15 punktów), co zwiększa wydajność. System obsługuje błędy poprzez timeouty, ponowne próby i wykrywanie rozłączeń. Robot potwierdza wykonanie komend odpowiedzią "OK".

5 Szczegółowa struktura kodu

5.1 Pliki główne aplikacji

5.1.1 `simple_gui.py` - Interfejs użytkownika

Lokalizacja: Główny folder projektu

Główne klasy i funkcje:

- `SimpleRobotGUI` - Główna klasa interfejsu
- `toggle_connection()` - Zarządzanie połączeniem z robotem
- `_handle_command()` - Obsługa komend głosowych
- `convert_to_face_drawing()` - Konwersja przy pomocy AI
- `start_robot_drawing()` - Rozpoczęcie rysowania

Kluczowe zmienne konfiguracyjne:

- `robot_ip` - Adres IP robota (domyślnie "192.168.125.1")
- `robot_port` - Port prawego ramienia (domyślnie 1025)
- `robot_port_l` - Port lewego ramienia (domyślnie 1026)
- `max_x`, `max_y` - Wymiary obszaru roboczego (mm)
- `margin_x`, `margin_y` - Marginesy bezpieczeństwa (mm)

5.1.2 `robot_drawer.py` - Główny koordynator

Lokalizacja: Główny folder projektu

Główna klasa:

- `RobotDrawer` - Główny orchestrator systemu

Kluczowe metody:

- `connect()` - Nawiązanie połączenia z robotem
- `load_image()` - Wczytanie i przetworzenie obrazu
- `draw()` - Rysowanie pojedynczym ramieniem
- `draw_dual()` - Rysowanie dwoma ramionami
- `disconnect()` - Rozłączenie z robotem

5.1.3 robot_communication.py - Komunikacja TCP/IP

Lokalizacja: Główny folder projektu

Główna klasa:

- RobotController - Zarządzanie połączeniem TCP

Kluczowe metody:

- connect() - Nawiązanie połączenia socket
- send_move(x, y) - Wysłanie pojedynczego ruchu
- send_batch_moves() - Wysłanie kilku ruchów
- send_pen_up/down() - Kontrola długopisu
- send_start() - Rozpoczęcie rysowania

5.1.4 image_processor.py - Przetwarzanie obrazu

Lokalizacja: Główny folder projektu

Główne funkcje:

- load_and_process_image() - Główna funkcja przetwarzania
- detect_edges_threshold() - Detekcja krawędzi metodą threshold
- detect_edges_canny() - Detekcja krawędzi metodą Canny
- detect_edges_adaptive() - Adaptacyjna detekcja krawędzi
- find_contours() - Znajdowanie konturów
- simplify_contours() - Upraszczenie konturów

5.1.5 coordinate_transformer.py - Transformacja współrzędnych

Lokalizacja: Główny folder projektu

Kluczowe funkcje:

- transform_paths() - Główna transformacja współrzędnych
- master_slave_assign_contours() - Przydział konturów do ramion
- smooth_path() - Wygładzanie ścieżek
- optimize_path_order() - Optymalizacja kolejności (TSP)

5.1.6 voice_commands.py - Rozpoznawanie głosu

Lokalizacja: Główny folder projektu

Główna klasa:

- VoiceCommandListener - Obsługa komend głosowych

5.1.7 convert_to_lineart.py - Integracja z OpenAI

Lokalizacja: Główny folder projektu

Główne funkcje:

- generate_image_from_text() - Generowanie obrazu z tekstu
- convert_to_lineart() - Konwersja do na karykaturę lub portret

5.2 Pliki konfiguracyjne i dane

5.2.1 robot_gui_config.json - Konfiguracja aplikacji

Lokalizacja: Główny folder projektu

Przykład zawartości:

```
{
  "robot_ip": "192.168.125.1",
  "robot_port": "1025",
  "robot_port_l": "1026",
  "camera_index": 0,
  "use_center_origin": false,
  "enable_logo": true,
  "logo_size": 27,
  "enable_frame_filtering": true,
  "max_x": 290,
  "max_y": 210,
  "margin_x": 10,
  "margin_y": 10,
  "quality_var": "high",
  "detection_method": "adaptive",
  "enable_tsp": false,
  "use_batch_mode": true,
  "drawing_mode": "load",
  "brush_size": 20,
```

```
"dual_arm_mode": true,  
"text_prompt": "",  
"forbidden_buffer": 50  
}
```

5.2.2 line_art_catalog/ - Archiwum wygenerowanych obrazów

Lokalizacja: Główny folder projektu

Struktura folderów: YYYYMMDD_HHMMSS_style/

- Zawiera wszystkie wygenerowane obrazy
- Automatycznie tworzone podczas przetwarzania
- Przydatne do debugowania i archiwizacji

5.3 Pliki RAPID dla robota

5.3.1 Right_arm.mod - Program prawego ramienia

Lokalizacja: Główny folder projektu

Kluczowe parametry:

- Port: 1025
- Zakres pracy: pełny obszar (290×210 mm)
- Prędkość: v1500 mm/s

5.3.2 Left_arm.mod - Program lewego ramienia

Lokalizacja: Główny folder projektu

Kluczowe parametry:

- Port: 1026
- Zakazana strefa: (0,0) do (130,40) mm
- Prędkość: v1500 mm/s

6 Dostrajanie algorytmów przetwarzania obrazu

6.1 Parametry detekcji krawędzi

6.1.1 Threshold - parametry binaryzacji

Lokalizacja: `image_processor.py`, funkcja `detect_edges_threshold()`

Kluczowe parametry:

- `threshold_value` - Wartość odcięcia (domyślnie 127)
- `max_value` - Maksymalna wartość dla pikseli powyżej progu

Jak zmienić:

1. Otwórz plik `image_processor.py`
2. Znajdź funkcję `detect_edges_threshold()`
3. Zmodyfikuj wartość `threshold_value`

Wpływ na system:

- Niższa wartość - więcej wykrytych krawędzi
- Wyższa wartość - mniej wykrytych krawędzi
- Optymalna wartość zależy od oświetlenia zdjęcia

6.1.2 Canny - parametry detekcji

Lokalizacja: `image_processor.py`, funkcja `detect_edges_canny()`

Kluczowe parametry:

- `min_threshold` - Minimalny próg (domyślnie 50)
- `max_threshold` - Maksymalny próg (domyślnie 150)

Jak zmienić:

1. Otwórz plik `image_processor.py`
2. Znajdź funkcję `detect_edges_canny()`
3. Dostosuj wartości progów

Wpływ na system:

- Mniejsze różnice między progami - więcej detali
- Większe różnice między progami - mniej detali
- Dobiera się eksperymentalnie dla danego typu obrazu

6.1.3 Adaptive - parametry adaptacyjne

Lokalizacja: `image_processor.py`, funkcja `detect_edges_adaptive()`

Kluczowe parametry:

- `block_size` - Rozmiar bloku sąsiedztwa (domyślnie 11)
- `C` - Stała odejmowana od średniej (domyślnie 2)

Wpływ na system:

- Większy `block_size` - bardziej płynne przejścia
- Mniejszy `block_size` - bardziej szczegółowa detekcja
- Parametr `C` wpływa na czułość detekcji

6.2 Parametry upraszczania konturów

6.2.1 Parametr epsilon `approxPolyDP`

Lokalizacja: `image_processor.py`, funkcja `simplify_contours()`

Wartość domyślna: 0.01 (1% długości konturu)

Jak zmienić:

1. Otwórz plik `image_processor.py`
2. Znajdź wywołanie `cv2.approxPolyDP()`
3. Zmodyfikuj wartość epsilon

Wpływ na system:

- Mniejsze epsilon - więcej punktów, dokładniejsze kontury
- Większe epsilon - mniej punktów, prostsze kontury
- Zbyt małe epsilon spowalnia rysowanie
- Zbyt duże epsilon traci detale

6.2.2 Minimalna długość konturu

Lokalizacja: `image_processor.py`, funkcja `filter_contours()`

Wartość domyślna: 10 pikseli

Jak zmienić:

1. Otwórz plik `image_processor.py`
2. Znajdź warunek filtrowania konturów
3. Dostosuj wartość minimalnej długości

Wpływ na system:

- Mniejsza wartość - więcej małych detali
- Większa wartość - czystszy obraz, mniej szumu
- Zależy od rozdzielczości wejściowego obrazu

6.3 Parametry wygładzania ścieżek

6.3.1 Parametry krzywej Béziera

Lokalizacja: `coordinate_transformer.py`, funkcja `smooth_path()`

Jak zmienić:

1. Otwórz plik `coordinate_transformer.py`
2. Znajdź funkcję `smooth_path()`
3. Zmodyfikuj parametry interpolacji

Wpływ na system:

- Wygładza ostre kąty w konturach
- Poprawia płynność ruchu robota
- Może zmienić kształt detali

7 Modyfikacja współrzędnych i bezpieczeństwa robota

7.1 Parametry bezpieczeństwa dual-arm

7.1.1 Bufor kolizyjny

Lokalizacja: `simple_gui.py`, zmienna `forbidden_buffer`

Wartość domyślna: 40 mm

Jak zmienić:

1. W interfejsie przejdź do Settings
2. Znajdź sekcję "Dual-arm Settings"
3. Dostosuj wartość bufora

Wpływ na system:

- Mniejszy bufor - ramiona mogą pracować bliżej siebie
- Większy bufor - większa odległość bezpieczeństwa
- Zbyt mały bufor zwiększa ryzyko kolizji
- Zbyt duży bufor zmniejsza efektywność obszaru

7.1.2 Zakazana strefa lewego ramienia

Lokalizacja: `coordinate_transformer.py`, funkcja `master_slave_assign_contours()`

Współrzędne domyślne: (0,0) do (130,40) mm

Jak zmienić:

1. Otwórz plik `coordinate_transformer.py`
2. Znajdź definicję zakazanej strefy
3. Dostosuj współrzędne do geometrii robota

Wpływ na system:

- Musi odpowiadać rzeczywistej geometrii robota
- Zbyt mała strefa zwiększa ryzyko kolizji z podstawą
- Zbyt duża strefa zmniejsza użyteczny obszar

7.2 Parametry ruchu robota

7.2.1 Prędkości ruchu

Lokalizacja: Pliki RAPID (`Right_arm.mod`, `Left_arm.mod`)

Wartości domyślne:

- Rysowanie: `v1500 mm/s`
- Przemieszczanie: `v1500 mm/s`

Jak zmienić:

1. Otwórz plik RAPID odpowiedniego ramienia
2. Znajdź definicje prędkości
3. Dostosuj wartości (zachowaj ostrożność!)

7.2.2 Precyzja pozycjonowania

Lokalizacja: Pliki RAPID

Wartości domyślne:

- Strefa ruchu: `z0` (bardzo precyzyjna)
- Tolerancja: `0.1 mm`

Jak zmienić:

1. Otwórz plik RAPID
2. Znajdź definicje `move_zone`
3. Dostosuj do wymagań jakości

7.3 Parametry systemu współrzędnych

7.3.1 Początek układu współrzędnych

Lokalizacja: `simple_gui.py`, zmienna `use_center_origin`

Wartości: `True` (środek), `False` (róg)

Jak zmienić:

1. W interfejsie przejdź do Settings
2. Znajdź opcję "Use Center Origin"

3. Wybierz preferowany system

Wpływ na system:

- Center origin - początek w środku obszaru rysowania
- Corner origin - początek w lewym dolnym rogu
- Wpływa na interpretację współrzędnych w programach RAPID

7.3.2 Skalowanie współrzędnych

Lokalizacja: `coordinate_transformer.py`, funkcja `transform_paths()`

Jak zmienić:

1. Otwórz plik `coordinate_transformer.py`
2. Znajdź funkcję skalowania współrzędnych
3. Dostosuj współczynniki skalowania

Wpływ na system:

- Musi odpowiadać rzeczywistym wymiarom obszaru roboczego
- Błędne skalowanie powoduje błędy pozycjonowania
- Ważne przy zmianie rozmiaru papieru

8 Debugowanie i rozwiązywanie problemów

8.1 Debugowanie połączenia z robotem

8.1.1 Sprawdzanie dostępności robota

Narzędzia:

- Polecenie: `ping 192.168.125.1`
- Sprawdza dostępność kontrolera robota
- Diagnostyka problemów sieciowych

8.1.2 Testowanie portów komunikacyjnych

Lokalizacja: `robot_communication.py`

Jak debugować:

1. Dodaj logowanie w funkcji `connect()`
2. Sprawdź czy socket się otwiera
3. Testuj wysyłanie prostych komend

8.1.3 Debugowanie timeout'ów

Lokalizacja: `robot_communication.py`, stała `START_COMMAND_TIMEOUT`

Wartość domyślna: 90 sekund

Jak zmienić:

1. Otwórz plik `robot_communication.py`
2. Znajdź definicję timeout'u
3. Zwiększ wartość dla wolniejszych połączeń (ale to i tak mega dużo czasu, ale chodzi o pozycjonowanie go jeżeli robot by miał kiedyś np szukać sam kartki tak jak wcześniej robił)

8.2 Debugowanie przetwarzania obrazu

8.2.1 Wizualizacja kroków przetwarzania

Lokalizacja: `robot_drawer.py`, metoda `show_processing_steps()`

Jak włączyć:

1. Otwórz plik `robot_drawer.py`
2. Odkomentuj wywołanie `show_processing_steps()`
3. Uruchom przetwarzanie obrazu

8.2.2 Zapisywanie obrazów pośrednich

Lokalizacja: `image_processor.py`

Jak dodać:

1. W kluczowych miejscach dodaj `cv2.imwrite()`
2. Zapisz obrazy po każdym etapie przetwarzania
3. Porównaj z oczekiwanymi wynikami

8.3 Debugowanie ruchu robota

8.3.1 Rejestrowanie komend wysyłanych do robota

Lokalizacja: `robot_communication.py`

Jak włączyć:

1. Dodaj `print()` przed każdym `send_command()`
2. Loguj wszystkie wysyłane komendy
3. Sprawdź sekwencje ruchów

8.3.2 Testowanie pojedynczych ruchów

Lokalizacja: `robot_communication.py`, metoda `send_move()`

Jak testować:

1. Utwórz prosty skrypt testowy
2. Testuj ruchy w bezpiecznych współrzędnych
3. Sprawdź odpowiedzi robota

8.4 Debugowanie trybu dual-arm

8.4.1 Wizualizacja przydziału konturów

Lokalizacja: `coordinate_transformer.py`

Jak debugować:

1. Wizualizacja znajduje się w settings->input methods->open detailed path viewer->animate dual-arm paths

8.4.2 Testowanie synchronizacji ramion

Lokalizacja: `robot_communication.py`, metoda `draw_paths_dual()`

Jak debugować:

1. Dodaj logowanie sekwencji komend
2. Sprawdź czasy wykonania dla obu ramion
3. Testuj komendy WAIT synchronizacji

9 Rozszerzanie funkcjonalności systemu

9.1 Dodawanie nowych metod detekcji krawędzi

9.1.1 Implementacja nowej metody

Lokalizacja: `image_processor.py`

Szablon implementacji:

1. Dodaj funkcję `detect_edges_[nazwa]()`
2. Zaimplementuj algorytm detekcji
3. Dodaj obsługę w funkcji głównej `load_and_process_image()`
4. Zaktualizuj interfejs wyboru metody

9.1.2 Integracja z interfejsem

Lokalizacja: `simple_gui.py`

Jak dodać:

1. Dodaj nową wartość do `detection_method`
2. Zaktualizuj menu wyboru w Settings
3. Dodaj obsługę w funkcji przetwarzania

9.2 Dodawanie nowych komend głosowych

9.2.1 Rozszerzenie słownika komend

Lokalizacja: `voice_commands.py`

Jak dodać:

1. Dodaj nową komendę do słownika rozpoznawania
2. Zaimplementuj obsługę w funkcji w `simple_gui.py` w funkcji `_handle_command()`
3. Zaktualizuj dokumentację komend

9.2.2 Implementacja akcji komendy

Lokalizacja: `simple_gui.py`, metoda `_handle_command()`

Jak dodać:

1. Dodaj case dla nowej komendy
2. Zaimplementuj logikę działania
3. Dodaj obsługę błędów

9.3 Dodawanie nowych stylów rysowania

9.3.1 Implementacja stylu w OpenAI

Lokalizacja: `convert_to_lineart.py`

Jak dodać:

1. Dodaj nowy prompt dla stylu
2. Dostosuj parametry generowania
3. Testuj jakość wyjściową

9.3.2 Integracja z interfejsem

Lokalizacja: `simple_gui.py`

Jak dodać:

1. Dodaj przycisk dla nowego stylu
2. Zaimplementuj funkcję konwersji
3. Zaktualizuj wybór stylu